

Justification technique — Bascule modèle LLM

Numéro : ADR-0001

Équipe n° : 16 (APOCAL'IPSSI 2026)

Titre : Bascule latence génération de quiz : Ollama Llama 3.1 8B CPU -> Hybride Groq Instant (cloud) + Ollama Llama 3.1 8B GPU (repli)

Membres : Rania ZERAMDINI, Amani LAYOUNI, Badreddine CHEBBOUR, Taise de Thèse NGANGA YABIE, Awadi BEDJA MROINKOKO SAID, Clément BASTIEN

Statut : ☒ Accepté · ☐ Proposé · ☐ Déprécié · ☐ Remplacé

Version : v1.0

Date : 30/06/2026 (J2)

Statut : Prêt pour validation PO

Problème

En configuration CPU (Ollama Llama 3.1 8B, config initiale du projet), la génération d'un quiz de 10 questions à partir d'un cours uploadé prend 124.95s en médiane (min 112.57s, max 146.89s sur 5 runs). Aucune optimisation de prompt ne peut compenser un tel écart : le goulot est l'inférence elle-même, pas la qualité du prompt.

Un test utilisateur réel (étudiant M2) confirme l'impact : après ~45s d'attente sans retour visuel, l'utilisateur pense l'application cassée et est sur le point d'abandonner. Sur les 124.95s mesurés en CPU, l'abandon avant fin de génération est donc quasi certain, le problème n'est pas juste "lent", il est au-dessus du seuil de patience d'un utilisateur qui n'a aucun indicateur de progression.

Impact produit

- **Démo PO** : le sponsor a explicitement demandé un temps de génération acceptable ce soir même ; livrer à 125s revient à un rejet en démo, sans discussion possible.
- **Adoption** : un temps d'attente de cet ordre sur l'action centrale du produit (générer un quiz) tue l'usage avant même l'évaluation de la qualité des questions produites.
- **Effet cascade** : si rien n'est fait, le MVP Release 1 est compromis, avec un risque direct sur la suite du sprint (stories dépendantes non validables).

Le modèle initial n'était hébergé que pour un usage local CPU, sans anticipation de la charge réelle d'inférence sur un cours complet, d'où la nécessité de mesurer avant d'agir plutôt que de deviner une solution.

Décision

Bascule vers une architecture hybride cloud + local :

Rôle	Modèle	Latence p50	Gain vs baseline
Primary	Groq Llama 3.1 8B Instant	1.08s	99.1%
Fallback 1	Ollama Llama 3.1 8B (GPU)	6.85s	94.5%
Fallback 2	Groq Llama 3.3 70B	2.35s	98.1%
Fallback 3	Phi (GPU)	11.72s	91.3%

Bascule pilotée par feature flag LLM_BACKEND=groq|groq-llama33|ollama-gpu|phi-gpu, timeout 30s → basculement automatique en cascade.

Pourquoi ce choix

- **Pas de compromis qualité** : score qualité ~8/10, identique à la baseline CPU.
- **Groq seul insuffisant en robustesse** : dépendance API externe → fallback GPU local obligatoire pour continuité de service.
- **Ollama GPU seul insuffisant en vitesse** : 6.85s reste 6× plus lent que Groq ; ne suffit pas seul si Groq est disponible.
- **RGPD** : Groq ne conserve pas les données (suppression 24h), Ollama local 100% conforme → les deux options passent le filtre conformité.
- **Coût** : free tier Groq (< 1 USD/mois), pas d'impact budget.

Mesures à l'appui

15 benchmarks (5 runs × 3 configurations), sur machine identique, cours de référence identique. Détail protocole et tableau complet CPU/GPU/Cloud → voir ADR-01 §2.1-2.3.

Conséquences assumées

- Dépendance à une API cloud (Groq) → mitigée par fallback local GPU.
- Dépendance au GPU du poste → mitigée par fallback cloud.
- Complexité accrue de la stack (multi-fallback) → acceptée contre le gain de 99.1% de latence.

Suivi

Revue de l'ADR-01 prévue 10 jours après mise en production. KPI surveillé : latence p50 < 2s (alerte si > 3s), taux de succès Groq $\geq$ 95%.